GIT UND DATENBANKEN

Und was ist mit einer Datenbank?

Git und eine XML-Datenbank können gut zusammenarbeiten. Entscheidend ist, welche Aufgaben sie übernehmen und wo die verbindlichen Editionsdaten und ihre Geschichte liegen.

Ihr IT-Team schlägt eine XML-Datenbank vor. Brauchen Sie dann noch Git? Und verzichten Sie mit Git auf leistungsfähige Abfragen? Diese Vertiefung erklärt die Arbeitsteilung. Sie führt nicht in ein bestimmtes Datenbankprodukt ein.

> Im Albrecht-Team sollen alle Briefe mit unsicherer Datierung und einer bestimmten Personenreferenz gefunden werden. Das kommt immer wieder vor. Ein zentraler XML-Abfragedienst kann diese Arbeit erleichtern. Samir fragt zugleich: Auf welchem Datenstand lief die Abfrage – und wo halten wir die daraus folgenden Korrekturen fest?

Aufgaben

Änderungen festhalten

Git versioniert Dateien und zusammengehörige Zustände. Es kann zeigen, wie TEI, Schema und Verarbeitung gemeinsam verändert wurden.

Beispiel und Tipp

Eine korrigierte Datierung und die zugehörige Anpassung einer Prüfregel werden gemeinsam festgehalten.

XML untersuchen

XML-Werkzeuge und XML-Datenbanken verstehen die ausgezeichnete Struktur. XPath und XQuery helfen beim Nachschlagen, Prüfen und Auswerten. Git führt diese Abfragen nicht selbst aus.

Beispiel und Tipp

Eine Abfrage findet Briefe, deren Datierung unsicher ist und die auf dieselbe Person verweisen.

Publikationen herstellen

Transformationen und Anwendungen erzeugen Ansichten, Register und Suchmöglichkeiten. Das technische Frontend stellt ein Interface zur Erschließung der Edition bereit.

Beispiel und Tipp

Eine freigegebene TEI-Fassung liefert Webansichten und Register. Welcher Stand verwendet wurde, wird dokumentiert.

Ergebnisse erhalten

Versionierte Daten, Publikationsfassungen und Herstellungsressourcen werden zugeordnet und unabhängig gesichert. Ein laufender Dienst und eine Archivkopie erfüllen verschiedene Aufgaben.

Beispiel und Tipp

Ein Release verweist auf Daten, Programme und Webausgabe; der Archivdienst erhält das vereinbarte Paket.

Was bringt die Datenbank konkret?

Eine XML-Datenbank wie eXist-db kann einen gemeinsamen, indizierten Bestand für wiederholte strukturierte Abfragen und Anwendungen bereitstellen. Das ist wertvoll, wenn Kombinationen häufig wechseln, viele Beteiligte denselben Bestand untersuchen oder Abfragen auf Dateien nicht mehr zweckmäßig sind.

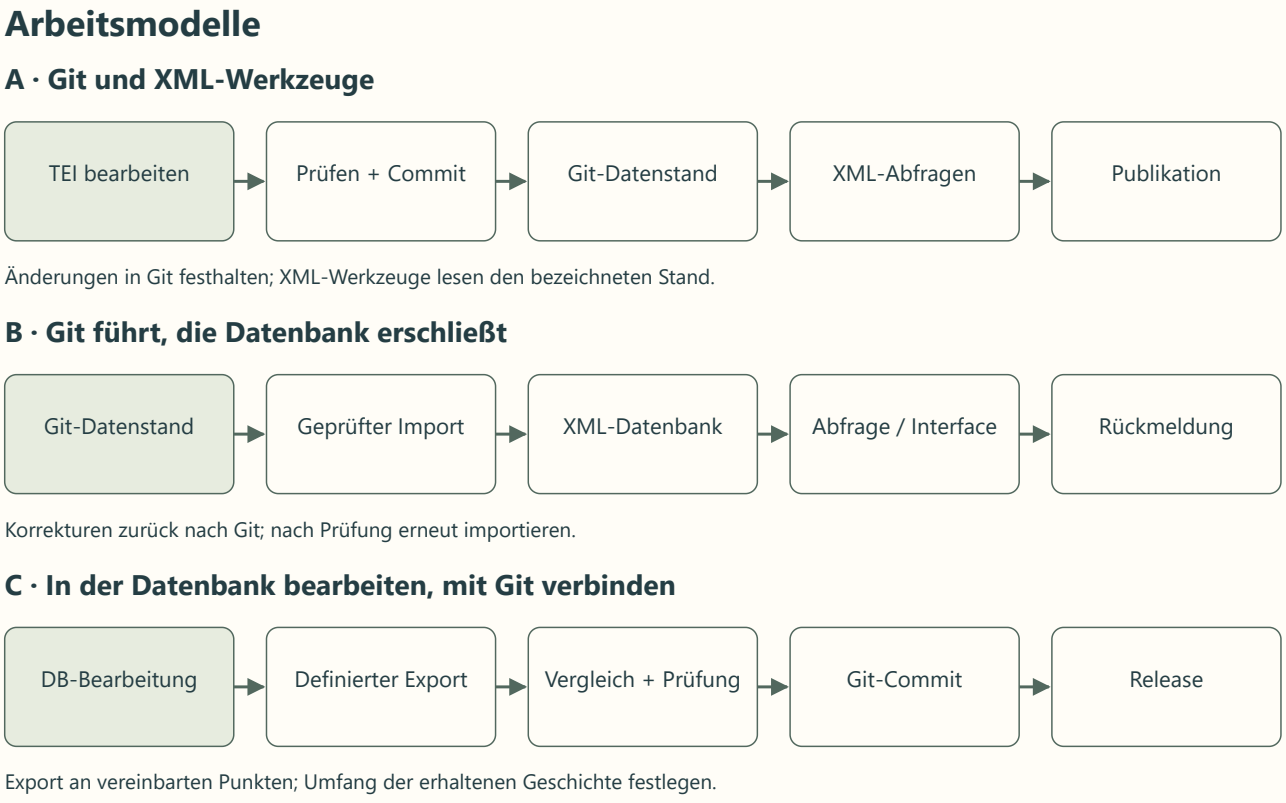
Für dieselbe editorische Frage können auch dateibasierte XML-Werkzeuge genügen, etwa oXygen oder Saxon. XPath bezeichnet Wege und Bedingungen in XML; XQuery kann Informationen aus XML-Dokumenten auswählen und zusammenführen. Ein Index hält vorbereitete Suchstrukturen bereit, die passende Zugriffe beschleunigen können. Die nötige XML-Kompetenz muss in beiden Arbeitsweisen verfügbar sein.

Nächster Schritt

Prüfen Sie drei reale editorische Abfragen und zwei gewünschte Webfunktionen am tatsächlichen Bestand. Vergleichen Sie Ergebnis, Bearbeitungszeit, Einrichtung und Pflege. Eine feste Dateizahl oder die Größe des Teams entscheidet nicht über den Datenbankbedarf.

So würden wir beginnen

Wir organisieren TEI-Dateien, Modell, Abfragen, Transformationen und Dokumentation als nachvollziehbaren Bestand in Git. XML-Werkzeuge ergänzen die Abfragefunktionen. Einen Datenbankdienst setzen wir ein, wenn er einen konkreten Nutzen bringt. Ein bereits betreuter institutioneller Dienst kann auch für ein kleines Projekt sinnvoll sein.



Schematische Darstellung · Arbeitsmodelle

Grafik als Text lesen

1. **Git und XML-Werkzeuge**: TEI bearbeiten → Prüfen + Commit → Git-Datenstand → XML-Abfragen → Publikation. Git ist der maßgebliche Bestand. Bearbeitende vergleichen und prüfen Änderungen, halten sie als Commit fest und übertragen sie in das gemeinsame Repository. XML-Werkzeuge lesen die Dateien. Die Veröffentlichung kann statisch erzeugt werden; auch eine laufende Anwendung ist möglich.
2. **Git führt, die Datenbank erschließt**: Git-Datenstand → Geprüfter Import → XML-Datenbank → Abfrage / Interface → Rückmeldung. Die maßgeblichen Editionsdaten liegen in Git. Ein geprüfter Import baut den Datenbankbestand auf. Die Datenbank bedient Abfragen und das Interface. Fachliche Korrekturen gehen über den vereinbarten Prüfweg zurück in Git; die Datenbank wird danach aktualisiert. Es gibt keine unkontrollierte zweite Schreibstelle.
3. **In der Datenbank bearbeiten, mit Git verbinden**: DB-Bearbeitung → Definierter Export → Vergleich + Prüfung → Git-Commit → Release. Eine vorhandene DB-Oberfläche bleibt die maßgebliche Bearbeitungsumgebung. Ein definierter Export übergibt geprüfte Änderungen an Git. Dort werden die

exportierten Daten und zugehörige Programme versioniert. Gleichzeitige Änderungen an exportierten Dateien und in der DB sind nicht ohne geregelte Rückführung zulässig.

A · Git und XML-Werkzeuge

Git ist der maßgebliche Bestand. Bearbeitende vergleichen und prüfen Änderungen, halten sie als Commit fest und übertragen sie in das gemeinsame Repository. XML-Werkzeuge lesen die Dateien. Die Veröffentlichung kann statisch erzeugt werden; auch eine laufende Anwendung ist möglich.

Beispiel und Tipp

Beispiel: Anna benötigt eine Übersicht unsicherer Datierungen. Eine dokumentierte XML-Abfrage über ihre Dateien genügt. Sie versioniert die Abfrage mit den Editionsdaten.

Tipp: Beginnen Sie mit einer wiederholbaren Abfrage. Notieren Sie Eingabeversion, Werkzeug und Aufruf. Prüfen Sie erst danach, ob ein zentraler Dienst zusätzlichen Nutzen bringt.

B · Git führt, die Datenbank erschließt

Die maßgeblichen Editionsdaten liegen in Git. Ein geprüfter Import baut den Datenbankbestand auf. Die Datenbank bedient Abfragen und das Interface. Fachliche Korrekturen gehen über den vereinbarten Prüfweg zurück in Git; die Datenbank wird danach aktualisiert. Es gibt keine unkontrollierte zweite Schreibstelle.

Beispiel und Tipp

Beispiel: Das Albrecht-Team untersucht in der Datenbank die Briefdatierungen. Ein Befund führt zu einer Korrektur im Git-Bestand. Erst der nächste geprüfte Import übernimmt diese Änderung in den Dienst.

Tipp: Datenstand, Importskript, Anwendung und Indexkonfiguration im Deployment-Nachweis verbinden. Eine leere Testinstallation daraus neu aufbauen. Wissenschaftliche Zusatzannotation, die nur in der DB entsteht, ausdrücklich übernehmen und sichern.

C · In der Datenbank bearbeiten, mit Git verbinden

Eine vorhandene DB-Oberfläche bleibt die maßgebliche Bearbeitungsumgebung. Ein definierter Export übergibt geprüfte Änderungen an Git. Dort werden die exportierten Daten und zugehörige Programme versioniert. Gleichzeitige Änderungen an exportierten Dateien und in der DB sind nicht ohne geregelte Rückführung zulässig.

Beispiel und Tipp

Beispiel: Das Team arbeitet bereits mit einem institutionellen XML-Dienst. Nach einem vereinbarten Bearbeitungsschritt exportiert es die Dokumente, prüft Löschungen, Referenzen und Vollständigkeit und hält die Änderung in Git fest.

Tipp: Exportanlass, Umfang, Verantwortliche und Fehlerbehandlung festlegen. Ein nächtlicher Export erhält Tagesstände, aber nicht automatisch alle Zwischenänderungen, Personenbezüge und Entscheidungsgründe.

Wie kommen wir an die Daten?

Ausgecheckte TEI-Dateien sind unmittelbar mit einem passenden Werkzeug zugänglich. Frühere Dateistände werden mit Git bereitgestellt. Der reguläre Zugriff auf den internen Datenbankbestand erfolgt über die Datenbanksoftware, etwa mit XQuery, API oder Export. Exportiertes XML ist danach ebenfalls unabhängig nutzbar. Prüfen Sie, welche Informationen und Beziehungen der Export tatsächlich umfasst.

Maßgeblicher Bestand

Der vereinbarte Ort, dessen freigegebene Inhalte für Übernahme und Publikation gelten. Andere Kopien dürfen nicht unbemerkt davon abweichen.

Build

Ein dokumentierter Verarbeitungslauf, der aus Daten und Regeln etwa HTML, Register und Suchindizes erzeugt.

Release

Eine bewusst freigegebene, bezeichnete Fassung. Git-Tags markieren den zugehörigen Commit; Archivierung ist ein weiterer Schritt.

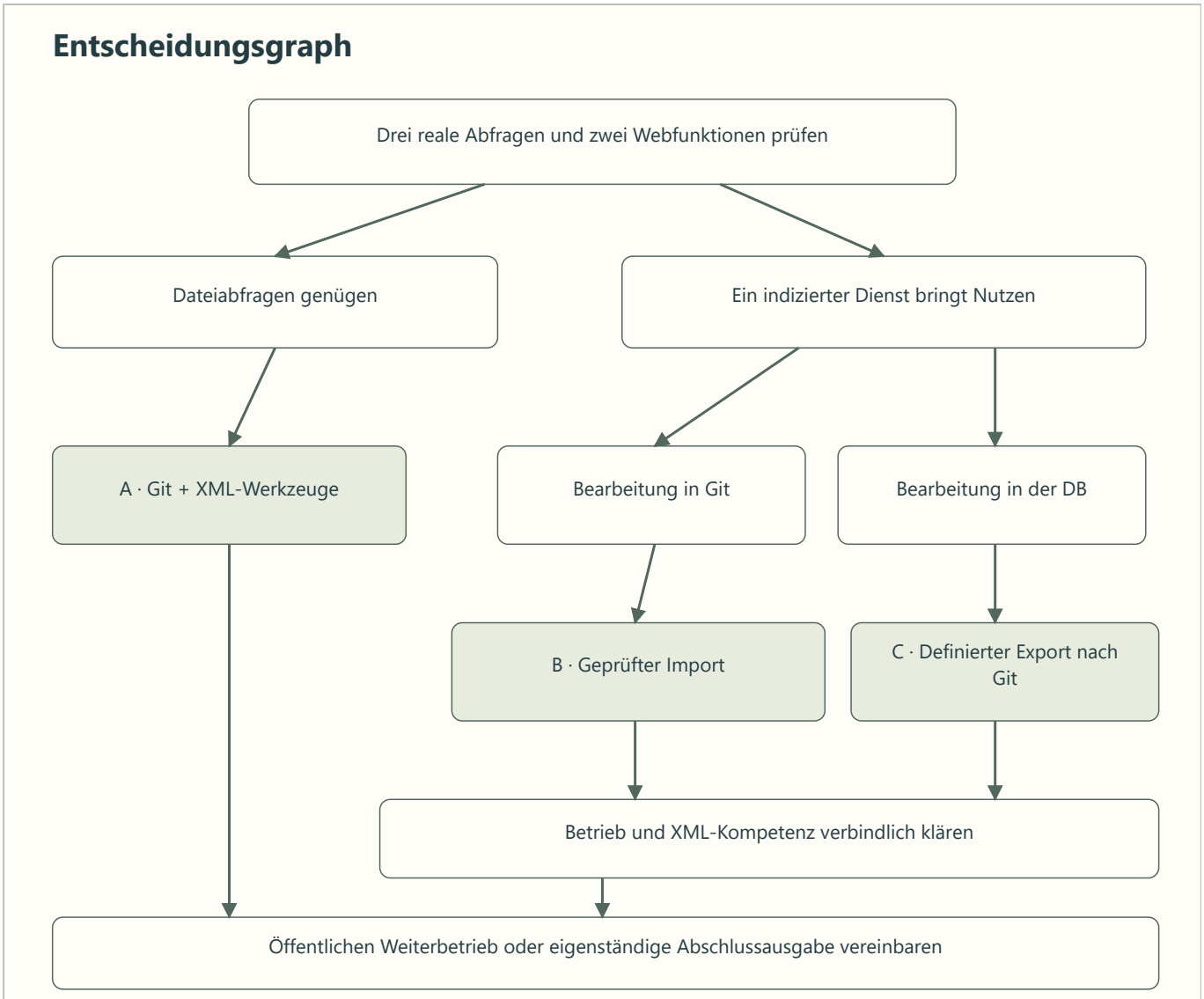
Warum wir die Geschichte erhalten

Eine spätere Redaktion fragt, warum Datierungen früher als unsicher galten. Die erhaltene Geschichte zeigt die frühere Auszeichnung, ihre Änderung und die dazugehörige Prüfregel. Begründungen werden in Commitnachrichten oder Entscheidungsnotizen ergänzt. So bleiben nicht nur Endergebnisse, sondern auch editorische Versuche und Einsichten untersuchbar.

Auch eine Datenbanklösung kann mit Versionierung eingerichtet werden. Eine zusammenhängende Geschichte der Änderungen an Texten, Schemata und Programmen entsteht dadurch jedoch nicht automatisch. Wir empfehlen, diese Geschichte mit Git zu erhalten und die Datenbank nach ihrem konkreten Nutzen einzusetzen.

> ## Nächster Schritt
>
> Erhalten Sie die relevante vollständige Git-Historie samt Modellen, Programmen und Entscheidungsdokumenten. Wer sie später lesen darf, wird gesondert vereinbart: Bewahren bedeutet nicht automatisch öffentlich freigeben.

Brauchen wir einen Datenbankdienst?



Schematische Darstellung · Entscheidungsgraph

Grafik als Text lesen

1. Welche Abfragen brauchen wir?: Noch nicht praktisch geprüft / Dateiabfragen genügen / Ein gemeinsamer indizierter Dienst bringt konkreten Nutzen
2. Wo werden die Editionsdaten bearbeitet?: In Dateien, mit Git / In einer vorhandenen DB-Oberfläche
3. Wer trägt einen Datenbankdienst?: Noch nicht zugesichert / Ein benannter Dienst mit zugesagtem Umfang
4. Muss er nach Projektende öffentlich laufen?: Nein, eine bezeichnete Ausgabe genügt / Ja, für eine notwendige dynamische Funktion

Unsere Orientierung: Bedarf zuerst praktisch klären

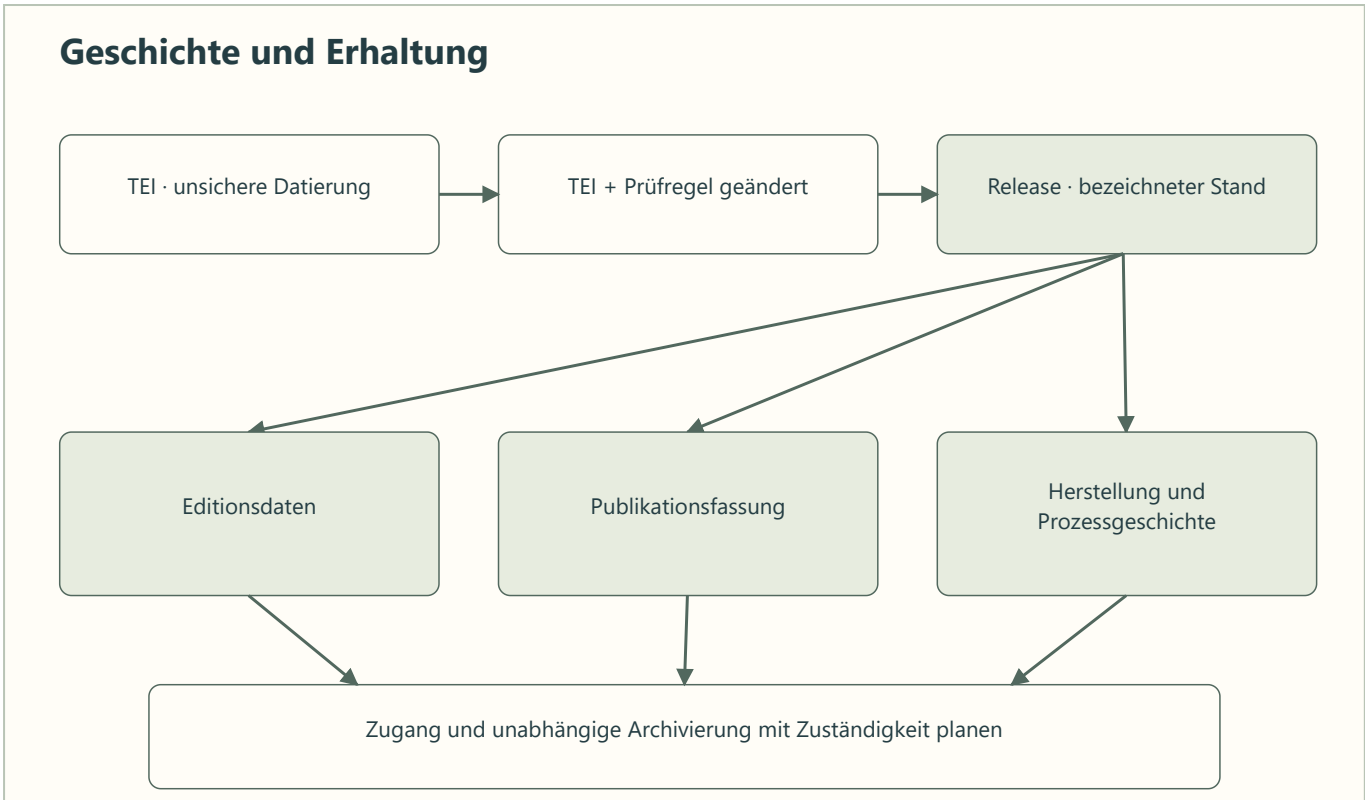
Beginnen Sie mit Git und einer dokumentierten XML-Abfrage. Prüfen Sie drei reale Abfragen und zwei Webfunktionen, bevor Sie einen zusätzlichen Dienst festlegen.

> Vor einem zusätzlichen DB-Dienst: Zuständigkeit, verfügbare XML-Kompetenz und Leistungsumfang verbindlich klären. Eine mögliche Installation ist noch keine Betriebszusage.

Prüfen Sie einen eigenständig nutzbaren Publikationsstand. Eine Datenbank kann während der Arbeit helfen, ohne dauerhaft Teil der öffentlichen Ausgabe zu bleiben.

Was bleibt nach Projektende?

Drei Bestände werden auf denselben bezeichneten Stand bezogen. Die öffentliche Bereitstellung und die unabhängige Archivierung werden jeweils mit Zuständigkeit geplant.



Schematische Darstellung · Geschichte und Erhaltung

Grafik als Text lesen

1. **Editionsdaten**: TEI, Metadaten, Verknüpfungen, Modell, Dokumentation und Rechte.
2. **Publikationsfassung**: Die konkrete Webausgabe, Register, Suchindex und gegebenenfalls Druck-/PDF-Ausgabe. Unverzichtbare externe Dienste benennen.
3. **Herstellung und Prozessgeschichte**: Relevante Git-Historie, Abfragen, Transformationen, Tests, Konfigurationen, Abhängigkeiten und Entscheidungsnotizen.

Editionsdaten

TEI, Metadaten, Verknüpfungen, Modell, Dokumentation und Rechte.

Publikationsfassung

Die konkrete Webausgabe, Register, Suchindex und gegebenenfalls Druck-/PDF-Ausgabe. Unverzichtbare externe Dienste benennen.

Herstellung und Prozessgeschichte

Relevante Git-Historie, Abfragen, Transformationen, Tests, Konfigurationen, Abhängigkeiten und Entscheidungsnotizen.

Merken

Statisch oder dynamisch ist eine eigene Entscheidung. Git kann eine laufende Anwendung versorgen; eine Datenbank kann statische Ausgaben erzeugen. Ein Git-Repository ersetzt weder einen Archivdienst noch die Finanzierung eines laufenden Systems.

Für das IT-Gespräch und den Antrag

Ihre Angaben bleiben im Sitzungsspeicher dieses Tabs. Der Sprachwechsel übernimmt sie. Bei gesperrtem Speicher können Sie das Blatt weiterhin nutzen und exportieren.

Drei reale XML-Abfragen und ihr Zweck

Zwei benötigte Webfunktionen; was lässt sich vorab berechnen?

Maßgeblicher Datenbestand und erlaubte Schreibstellen

Import/Export: Anlass, Umfang, Prüfung und Fehlerbehandlung

Welche Zwischenstände und Entscheidungsgründe erhalten wir?

Zuordnung von Daten, Anwendung, Konfiguration und Veröffentlichung

Zuständige Rollen, XML-Kompetenz und zugesagte Leistungen

Abschluss, Weiterbetrieb und Archivierung: Wer übernimmt was?

Nächster praktischer Test und eingeplantes Budget

Formulierungsbausteine für den Antrag

Git-basierte Grundarchitektur

Wir führen die Editionsdaten, das dokumentierte Modell und die Herstellungsressourcen in Git. Veröffentlichungen werden einem bezeichneten Daten- und Werkzeugstand zugeordnet. Für Qualitätsprüfung, Abschluss und institutionelle Übergabe sind [Rolle/Budget] vorgesehen.

Zusätzlicher Datenbankdienst

Für [konkrete Abfragen/Funktion] nutzen wir [Dienst]. Maßgeblicher Datenbestand, Änderungsweg und Wiederaufbau sind dokumentiert. [Verantwortliche Stelle] übernimmt [Leistungsumfang/Laufzeit]; Weiterbetrieb oder Abschlussalternative und Archivübergabe sind vereinbart.

In der Strategie-Werkstatt weiterarbeiten →

Weiter zu XML-Datenbanken und Werkzeugen

Hier steht die Git-Strategie im Mittelpunkt. Die offiziellen Quellen erläutern eXist-db und XML-Werkzeuge ausführlicher, einschließlich Community-Angeboten und professioneller Unterstützung. Produkteditionen und zugesagte Leistungen prüfen Sie für Ihr konkretes Projekt.

- eXist-db · Documentation
- eXist-db · XQuery

- eXist-db · Community / professional support
- Saxonica · Using XQuery
- Git · Documentation
- TEI · Guidelines

Redaktionelle Grundlage: die beiden bereitgestellten Arbeitsdokumente zu Git und XML-Datenbanken. Text, Übersetzung, schematische Grafiken und Umsetzung wurden KI-unterstützt erstellt. Version 1.0 · 28.10.2026. Die Grafiken zeigen Arbeitsmodelle, keine reale Installation.